TrainCache P1 实施指南 v4

P1 实施指南：训练数据流加速系统（代号 TrainCache）— v4 重构版

1. 项目介绍
2. 问题与动机（含 2026-10 生态现勘）
   2.1 外部事实（2026-10-05 核实，这些事实构成立项的边界）
   2.2 仍然成立的问题（这就是本项目要钻的缝）
3. 架构总览
4. 阶段 0：现勘（与使用阶段合并，1–2 周，闸门制）
   4.0 可行性探针（最先做，2–3 天，在写任何控制器代码之前）
5. 基座设计
   5.1 CRD 设计（v4 status 字段增量）
   5.2 双层标识模型（保留 v3 不变）
   5.3 缓存 Agent（v4 增量：加两个新职责模块）
   5.4 读侧信号与命中率口径（v4 把 inotify 同时用作消费顺序校准）
6. 核心 1：训练全程数据流加速闭环（项目灵魂，v3 → v4 升级）
   6.1 三阶段统一控制回路
   6.2 前缀就绪度形式化定义与得分公式（v4 核心修正）
   6.3 调度-启动预取阶段（v4 保留 v3 主体，得分公式切换）
   6.4 训练中持续预取阶段（v4 全新）
   6.5 阶段切换重预取阶段（v4 全新）
   6.6 超时降级（v3 保留 + v4 新增全程降级）
   6.7 预注册假设与盈亏平衡旗舰图（v3 → v4 扩展）
7. 核心 2：欠配缓存下的准入与预取（v3 → v4 定位降级，保留主体）
   7.1 为什么 v3 的 W-TinyLFU 旗舰图可能是一条平线
   7.2 实验设计
   7.3 与 §6.4 持续预取的协同（v4 新增）
8. 支撑臂设计（v3 保留，无改动）
   8.1 S3 合并回源网关（保留 v3）
   8.2 P2P 块共享：在 CE 基线上复现 EE”缓存组”语义（保留 v3）
   8.3 Checkpoint 异步 sidecar（保留 v3，语义错误已修正）
   8.4 混沌套件（v3 四件套 + v4 新增一件）
9. 调研/实验臂（只做实验或只写文档，绝不进主路径）
   9.1 RDMA 传输臂（保留 v3）

9.2 io_uring / S2（保留 v3，加 v4 持续预取路径上的预期收益注释）
9.3 版本化（保留 v3）
9.4 GPU 侧技术的知识边界（保留 v3）
10. Benchmark 方法学（v4 主要更新：分启动阶段与全程阶段两份报告）
10.1 三轨负载（v3 保留 + v4 加 D 轨）
10.2 在笔记本上造出真实的冷热差（v3 保留）
10.3 对照组矩阵（v4 把 v3 的五组扩展为按阶段报告）
10.4 硬纪律（v3 保留 + v4 加一条）
11. 项目优点（v4 更新）
12. 诚实清单（v4 更新）
13. 砍项清单（v3 → v4 变更摘要）
14. 实施步骤（v4 重新编排，14 步对应新核心）
15. 排期与死线（v4 倒推，与 P2 共用日历）
16. 预期成果（v4 更新）

P1 实施指南：训练数据流加速系统（代号 TrainCache）— v4 重构版

> 2026-10-13 v4 重构（学员在动手实现核心库与 Agent 服务化架构文档后定稿）： v3 的”预热-调度闭环”叙事只覆盖了 Job 从提交到第一个 step 这一启动阶段， 写好的一套块粒度缓存库只用了启动这一次。v4 把项目定位从”启动预热器” 升级为”训练全程数据流加速器”：把一次性的启动预热扩展成三阶段闭环 —— 调度时的启动前缀预取、训练中的按消费顺序持续预取、阶段切换时的重新预取。 得分公式从”覆盖率 readyRatio”改为”前缀就绪度 + 启动窗口加权”， 消除”预热了第 90 块但第 50 块没到、得分却虚高”的语义错位。
>
> v3 的资产（探针、现勘、bench 装置、缓存核心库 M1–M7 已落地的接口、 Agent 服务化架构文档现有的 gRPC/重扫描/inotify/命中率口径小节）一律保留， 本版只在三处做增量： §5.1 给 CRD status 加前缀就绪度字段、 §5.3 给 Agent 加 “持续预取”与”阶段切换重预取”两个新职责、 §5.4 把 inotify 同时用作消费顺序输入。 配套文档：`07-开源贡献指南.md` （回馈目标仍是 Fluid）。

1. 项目介绍

电梯陈述：这个项目要做一套面向 K8s 训练 Job 的**全程数据流加速器**。工业界的现有方案把数据预热做成了启动时的一次性手动动作，预热完之后就撒手不管——训练 Pod 落到节点上之后，整个 epoch 中后期出现的 data stall 没有人兜底。我们要把”启动预取”扩展成贯穿训练全程的闭环：调度阶段按 Job 实际会消费的开头块序列做前缀预取并用前缀就绪度门控；Pod 落地之后由 Agent 按消费顺序持续预取后续块；epoch 切换或 shuffle 重排时按新顺序重新预取。最后，我们用端到端 makespan 和真实部署起来的 Fluid/JuiceFS 同台跑分，分别量化启动阶段收益与全程收益。

三重定位（与 P2 同构，三者缺一不可）：

1. **对标者**：我们把 Fluid（Dataset/DataLoad/Runtime 加缓存亲和调度）和 JuiceFS 社区版当作一等对照组真实部署起来，在同样的负载下跑分，绝不打空气靶；
2. **测量者**：我们最看重的指标有四个——Job 从提交到跑起第一个 step 的时间、整个 epoch 的 makespan、训练全程的 dataloader stall 占比、缓存命中率分阶段曲线；预热时重复搬运的字节成本仍然计入分母；我们允许阴性结论，并且在实验开始之前就把假设预注册下来；
3. **改进者**：现勘中确认下来的缺口要用测量数据撑住，再向 **Fluid（Apache-2.0 许可、Go 语言，调度和预热正好落在它的代码域里）** 提 issue 和 PR，不能只停留在”我自己的仓库里有这个功能”。

这个项目明确不做什么：它不会再做一个分布式文件系统（我们不重造 JuiceFS 的 POSIX/FUSE 数据面）；它也不是一篇缓存算法论文（W-TinyLFU 在我们这里只充当工程对照，并不构成学术声称）；它全程都不需要 GPU——整个项目在 CPU 训练循环上就可以测，我们测的是 dataloader stall，跟 GPU 利用率没有关系；它不调度 epoch（epoch 是 Pod 内部循环，不触发 K8s 调度），阶段切换是节点内缓存行为，永远不进调度路径。

2. 问题与动机（含 2026-10 生态现勘）

2.1 外部事实（2026-10-05 核实，这些事实构成立项的边界）

- **Fluid 的缓存亲和调度，本质上是”粗粒度的二进制判断加事后注入”**：mutating webhook 在 Pod 创建的时候读取它引用的 Dataset，然后往 Pod spec 里注入 `nodeAffinity`（ACK 增强文档给出的默认权重为：缓存节点 100、同可用区 50、同地域 20，给节点打标签还可以升级成硬亲和），另外还有 `RequireNodeWithFuse` 与 `PreferNodesWithoutCache` 两个策略；如果 Pod 自己带了 affinity 或 nodeSelector，webhook 就会跳过注入。它能回答的问题只有”节点上**有没有**这个 Dataset 的 runtime/缓存”，**完全不知道缓存了多少比例、具体缓存了哪些块、缓存的开头几块在不在线**。
- **预热（DataLoad）是一个独立的、需要手动创建的 CRD**，它和调度决策之间没有闭环：DataLoad 不会因为”某个 Job 马上要被调度”而触发，调度器也不会根据 DataLoad 的块级进度来做门控。开源 Fluid 与 ACK 托管增强版之间的能力边界，你必须在现勘阶段划清楚。
- **JuiceFS 社区版 v1.3.1 采用 Apache-2.0 许可**：它支持 4MiB 块、warmup、`--prefetch` 预读和单机本地缓存；**分布式缓存组（一致性哈希环、peer 之间共享块）、IORPC、RDMA 都属于企业版能力**（EE 5.2/5.3 及以上版本）。本项目使用的 4MiB 块大小与社区版默认值对齐，这一点会作为报告注脚写明。
- **2026 年训练数据加载的主流形态**：webdataset tar 分片、Mosaic MDS 这类百 MB 级分片，配合确定性 shuffle 做流式读取；百万小文件的 ImageNet 模式现在只属于元数据压力场景，已经不再是 LLM 和大模型训练的典型 I/O 形态。
- **训练阶段切换的真实形态**：一个训练 Pod 从启动到结束**常驻同一个节点**，从 epoch 0 跑到最后一轮；epoch 切换、shuffle 重排、curriculum 阶段切换全部是 Pod 内部循环，根本不触发 K8s 调度。所以”阶段切换预热”如果要做，必须做成节点内的缓存预取行为，做成”重新调度 Pod 来预热”是设计错误。
- **数据并行下的分片对应关系**：分布式训练的主流形态是数据并行，整份数据被所有 Pod 合起来跑一遍，每个 Pod 只跑自己的分片，Pod 之间不重复。所以预热要按”Pod 分片”对应到候选节点，而不是每个候选节点都预热全量。

2.2 仍然成立的问题（这就是本项目要钻的缝）

1. **训练全程的数据流无人加速**：工业方案把预热当成启动时的一个动作，预热完之后训练 Pod 就靠自己读 page cache 和回源拉数据。epoch 中后期 page cache 失效、shuffle 之后顺序被打乱、热子集被换出，这些都会再次造成 data stall，但这时候 Pod 已经在节点上跑着，没有任何组件再帮它把数据提前铺好。这是 v3 叙事里被忽略的最大缺口。
2. **预热与调度之间没有闭环**：工业方案的做法是”先手动建一个 DataLoad，然后靠二进制亲和碰运气”，这两步之间缺少一条完整的控制回路——“Job 即将被调度 → 判断该往哪几个候选节点预热它需要的那些块 → 用块级就绪度决定何时放行、如何打分 → 超时之后怎么降级”；
3. **就绪度得分用覆盖率是错的**：v3 用 `readyRatio = 已预热块数 / 需要总块数` 作为门控与打分依据，但训练是按消费顺序读块的，只有开头连续的那一段对启动有用。预热了第 90~99 块但第 50 块没到，启动照样卡住，可覆盖率得分却虚高。正确的口径应该是”前缀就绪度”——从第 0 块开始连续就绪到第几块；

4. **预热成本没有人记账**：往 2 到 3 个候选节点重复预热，要付出 2 到 3 倍的字节与带宽代价，但这笔代价在什么情况下值得花，目前没有公开的曲线。v4 把这笔账扩展成”启动预热字节 + 全程持续预取字节”两笔，分别记账。

立项闸门（规矩和 P2 一样）：§2.1 对缺口的判断，必须在阶段 0 由**开源 Fluid 的实测取证**来确认（包括 webhook 注入了什么内容、DataLoad 与调度之间的时序关系、Fluid 在训练全程会不会做任何持续预取）；如果现勘发现开源 Fluid 已经提供了全程持续预取，叙事就收缩为”复现与量化”，不许硬造缺口。

3. 架构总览



三阶段闭环（v4 的核心叙事）：

阶段	时机	触发方	优先级	失败兜底
调度-启动预取	Job 提交到 Pod 落节点	Controller 通过 SchedulingGate	高（阻塞门控）	超时降级，去 gate 正常调度
训练中持续预取	Pod 落地到训练结束	Agent 自治	低（可被启动预取打断）	预取跟不上就退化成回源读，不影响正确性
阶段切换重预取	epoch / shuffle / curriculum 切换	DataLoader 钩子上报事件	中（高于持续预取，低于新 Job 启动）	切换瞬间允许 stall 短暂回升，重新铺数据后回落

数据面约定（保留 v3 的决策）：我们采用 preheat 模型——Agent 把数据块落到 `/var/cache/traincache/<ds>/blocks/`，训练 Pod 通过 hostPath PV 只读挂载这个目录，读数据时直接走内核的 page cache，**Agent 完全不参与读 I/O 路径**，所以读路径上没有任何额外延迟。代价是读侧的信号只能靠旁路采集（见 §5.4）。FUSE read-through 降级为可选的 stretch 目标（Fluid 生态里已经有成熟的 FUSE/CSI，我们不重造）。

消费顺序的来源（v4 新引入的关键设计）：训练 Pod 里跑一个轻量的”消费顺序生成器”sidecar（或 DataLoader 内的钩子），按 Job 配置（dataset、epochs、shuffle_seed、batch_size、prefetch_factor）算出当前阶段按消费顺序排列的块列表，通过共享内存或 Unix socket 喂给同节点的 Agent。Agent 拿到顺序之后，把启动窗口（默认前 K 块，K = batch_size × prefetch_factor）作为高优先级队列送进抓取管线，剩余块作为低优先级持续预取队列。详细接口见 §6.3。

4. 阶段 0：现勘（与使用阶段合并，1–2 周，闸门制）

> 这一节内容 v3 已经落定并已由学员执行完毕，相关产物在 `Project/TrainCache/probes/`、`survey/`、`bench/` 下。 v4 对现勘只追加一项**新增探针 C**：Fluid 在训练全程上有没有任何持续预取行为。 该探针结果决定 §6.3 的”训练中持续预取”叙事是否成立——若 Fluid 已有同类机制， 叙事收缩为”复现与量化”，不硬造缺口。其它现勘动作清单、闸门产出保持 v3 不变。

4.0 可行性探针（最先做，2–3 天，在写任何控制器代码之前）

这一步的目的，是先用几十行脚本证明实验方法论站得住脚——如果实验装置本身造不出可信的差异，三个月后再回头改设计，赔上的就是整个项目。这三天是整个 P1 性价比最高的三天。

- **探针 A：验证冷热差稳定且可复现**。准备同一批百 MB 分片，用 `tc qdisc ... tbf rate 100mbit burst ... netem delay 20ms` 挂在 Agent 到 MinIO 的路径上；冷读（先执行 `sync; echo 3 > drop_caches` 并清空本地缓存，读请求走 tc 链路）与热读（命中本地缓存，不经过 tc）各跑至少 7 次。
  - **绿灯标准**：冷读与热读的中位延迟相差至少 5 倍（或者中位吞吐差一个数量级）；两组的 bootstrap 95% 置信区间**互不重叠**；组内变异系数 CV 不超过 15%；
  - **如果亮了红灯，正确的做法是去排查装置本身，不要硬调参数去凑数字**：tc 是不是挂在了正确的网卡和出口上（用 `tc -s qdisc` 观察计数有没有在涨）、热读是不是真的零网络流量（用 `ss` 或 tcpdump 验证没有建连）、单块磁盘自己是不是已经成了瓶颈、MinIO 自己的 page cache 会不会制造出”假冷读”；
- **探针 B：验证 stall 指标能打出来**。搭一个最小的 CPU 版 PyTorch DataLoader（用 webdataset 读真正的 JPEG tar，`num_workers` 取 0/4/8 三档，不挂模型或只挂极小的矩阵乘），在训练循环里打点记录 `wait_time/step_time`。
  - **绿灯标准**：冷态下 stall 占比稳定高于 40%，热态下低于 15%；增加 workers 数量之后冷态 stall 单调下降（打点噪声造不出这种单调下降，所以它能证明我们测到的是真实的 I/O 饥饿）；
- **探针 C（v4 新增）：验证训练全程的 stall 是真实存在的**。在探针 B 的基础上让训练循环连续跑 3 个 epoch（每个 epoch 用确定性 shuffle，但 seed 不同），打点记录每个 epoch 内 stall 占比的时间序列。

    - **绿灯标准**：epoch 0 的中段（已过启动、未到结束）和 epoch 1 刚切换后的若干 step 都观察到 stall 显著回升（至少回升到冷态的 30% 以上）；如果中段和阶段切换都不回升，意味着冷读只发生在启动一次，“全程持续预取”叙事就要收缩；
    - 这一探针的目的是堵住”训练中根本没 stall、预取没必要”的反驳路径，是 v4 立项的关键闸门；
- **硬性时间盒为 3 天**。如果到第 3 天晚上仍然亮红灯，就停止开工，复盘根因（必要时上外接 SSD，或者提前租两台带独立数据盘的 CPU 云主机），**绝不带着毛病进入步骤 1**。

动作清单：

1. 搭好版本不低于 1.26 的 kind 集群（这是 PodSchedulingGate 的前置要求），在上面部署 MinIO、**JuiceFS 社区版（本地缓存模式）**和**开源 Fluid + JuiceFSRuntime**；
2. 跑通官方 demo：创建 Dataset、用 DataLoad 手动预热、启动带 dataset 标签的训练 Pod；抓取并归档 **webhook 实际注入的 Pod spec**（nodeAffinity 权重、FUSE 注入情况），以及 DataLoad 与 Pod 调度之间的**时序关系**（Pod 会不会等 DataLoad 完成？还是先被调度、再等数据？）；
3. **追加 v4 现勘动作**：跑通官方 demo 之后让训练 Pod 连续跑 3 个 epoch，观察 Fluid 的 JuiceFSRuntime 在 epoch 切换时会不会主动预热新顺序的数据（看 Runtime pod 的日志、`kubectl get dataset` 的 status 字段变化、tc 计数有没有回升），把结论写入现勘报告；
4. 整理一张能力边界表，把开源 Fluid、ACK 文档中的增强能力、JuiceFS 社区版、JuiceFS 企业版（缓存组/RDMA）四方放在一起对比；每一个判断都要附上版本号和证据（kubectl 输出、文档链接或截图）；
5. 用 JuiceFS 的 `warmup` 完成第一次”对手热读”基线，记录版本号与挂载参数；
6. 产出 **《P1 对手现勘报告》**（这份报告和 P2 的现勘报告合并为同一时期的总交付物）。

闸门产出：①如果缺口被证据确认，核心 1 就按”三阶段闭环”叙事推进；②如果缺口不成立（特别是 Fluid 已有全程持续预取），核心 1 改为”Fluid 机制的独立复现与参数量化”，不许带着假设开工。

5. 基座设计

> 基座层 v3 已基本落地。`core/` 下的 objectstore/cas/fetch/index/internal/digest 五个包 已完成 M1–M7 共 54 个单元测试全绿，接口签名冻结（见 `Project/TrainCache/docs/7.1 design.md`、`7.2 plan.md`、`7.3 decisions.md`）。v4 对基座只做三处增量： ① §5.1 给 CRD status 加前缀就绪度字段； ② §5.3 给 Agent 加”持续预取”和”阶段切换重预取” 两个新职责模块； ③ §5.4 把 inotify 同时用作消费顺序的旁路校准信号。 其它内容（路径索引、CAS 存储层、Agent 的 gRPC 接口、重扫描、inotify 命中率口径、辅助预热） 全部保持 v3 不变，本节只复述要点并指出增量位置。

5.1 CRD 设计（v4 status 字段增量）

```yaml
apiVersion: traincache.io/v1alpha1
kind: Dataset
spec:
  source: { endpoint: http://minio:9000, bucket: datasets, prefix: shards/ }
  chunking: { strategy: fixed, chunkSize: 4MiB }   # 与 JuiceFS 默认 block 对齐
  index: { algo: path-index }        # 索引层：路径寻址（见 5.2）
  storage: { digest: xxh3-128, dedup: true }        # 存储层：CAS
  version: v5                        # 版本前缀目录，不做自研失效协议（见 §9.3）
status:
  totalChunks: 262144
  nodes:                             # 粗粒度状态，细清单不下发 etcd
    - { node: node1,
        readyRatio: 0.98,           # 覆盖率：保留作粗略进度展示
        prefixLen: 8192,            # v4 新增：当前快照下从第 0 块起连续就绪的块数
        startupWindowReady: 0.96,  # v4 新增：启动窗口（前 K 块）的就绪比例
        bytes: 1.0Ti, inflightBytes: 64Gi }
  conditions: [{ type: Ready, status: "False", reason: Preheating }]
```

- **etcd 对象有 1.5MB 的上限**：块级清单放在对象存储上的 index shard 里（每个 shard 不超过 10 万条），CRD status 只存每个节点的 `readyRatio`、`prefixLen`、`startupWindowReady` 加上 shard 位图摘要（roaring bitmap 序列化之后的摘要哈希）；调度插件需要精确清单的时候，再按需拉取对应的 shard；
- **Job 的子集引用**：训练 Job（尤其是多 rank 的 Job）通过 label 或 annotation 声明自己要消费的 manifest 子集（分片列表）；**v4 新增**：子集声明里允许带 `consumeOrderRef` 字段指向一份预先算好的消费顺序文件（顺序生成器 sidecar 产出，见 §6.3）；就绪度一律按**子集**计算，不按整个数据集计算；
- **v4 新增的 status 字段语义**：
  - `readyRatio`：保留作为粗粒度进度展示，对外（`kubectl get`）容易看；
  - `prefixLen`：调度门控与打分的真实依据，单调不减（重启后由 Agent 重扫描回填）；
  - `startupWindowReady`：调度阶段的快速度量，等于 `min(prefixLen, K) / K`，是 `prefixLen` 在启动窗口内的归一化形式；
- `version` 字段就是源前缀的代次（类似 `shards/v5/` 这样的前缀目录）；index 构建支持增量：ETag 没有变化的 shard 直接跳过，避免对百万对象做全量 List。

5.2 双层标识模型（保留 v3 不变）

v3 的双层标识模型在 v4 完全保留：

- **索引层（决定取什么）**：manifest 条目键为 `(path, chunkIndex)`，采用路径寻址，不承诺跨路径去重；
- **存储层（决定怎么落盘）**：块在预热下载的时候本来就要流过内存，我们趁这个机会流式计算 xxh3-128 摘要作为存储键，落盘到 `blocks/ab/<contentHash>.blk`，并维护一份 refcount 索引；
- 读回校验、安全去重、refcount 归零才进入可驱逐集合等机制全部保留；
- 哈希选型上，内部信任边界里我们用 xxh3-128 换取性能，全量 SHA-256 校验保留为开关。

详细签名见 `core/docs/7.1 design.md` §5.2，v4 不改。

5.3 缓存 Agent（v4 增量：加两个新职责模块）

> Agent 服务化架构文档 `Project/TrainCache/docs/8.Agent服务化.md` 现有的 gRPC 接口、 /metrics 监控、启动重扫描、inotify 热度统计与命中率口径、辅助预热几节**全部保留不变**。 v4 在该文档末尾追加两个新功能点（持续预取 + 阶段切换重预取）， 本节给出形式化定义与接口签名，详细落地见 Agent 服务化文档的 v4 增量章节。

并发模型（保留 v3）：

```
抓取管线 = workqueue(块, 优先级) → errgroup + 按字节加权信号量 → 分块写盘
限制 bytes-in-flight（默认 512MiB）而非 goroutine 数：防小文件 goroutine 爆炸、大文件 OOM。
```

- HTTP 层的 `MaxConnsPerHost`（默认 64）与 MinIO 的连接池对齐；抓取优先级从高到低**v4 修订为四档**：显式启动预取、阶段切换重预取、邻域预取（训练中持续预取的一种）、后台补全；
- **原子可见性**：先写 tmp 文件再 `rename()`；缓存块不逐块 fsync——这个取舍和存储系统的直觉相反，但它是正确的；
- **驱逐策略**：采用双水位（占用到 85% 触发驱逐，一路回收到 75%），按 LRU 顺序驱逐，同时用 refcount 把正在读的块钉住； `unlink` 一个已经打开的文件是安全的，所以驱逐不需要等读者结束；配额按 Dataset 分组；
- **启动重扫描**：Agent 重启之后扫描 blocks 目录重建 CAS 索引，对块头抽样校验 1%，全量校验放在后台以低优先级进行——这是混沌韧性（§10.4）的地基。

v4 新增职责模块一：训练中持续预取（Persistent Prefetch）

```go
// PersistentPrefetcher 是 Agent 内部的预取编排器，独立于 gRPC 预热接口。
// 它消费 OrderSource 推送的消费顺序，按顺序把后续块送进抓取管线。
type PersistentPrefetcher struct { /* 持有 fetch.Fetcher、cas.Store、优先级队列、状态机 */ }

// Start 启动预取循环：从当前消费位点（cursor）开始，按 consumeOrder 把后续块
// 以"邻域预取"优先级送进 fetch.Fetcher.Fetch；每完成一块推进 cursor 并上报 /metrics。
// 启动预取（来自 gRPC Preheat 接口）和阶段切换重预取可以打断本循环，
// 打断时在途请求不取消（已花掉的带宽不要浪费），新请求改用更高优先级入队。
func (p *PersistentPrefetcher) Start(ctx context.Context) error

// OnConsumeSideAdvance 由 DataLoader 钩子（OrderSource 实现者）调用：
// 训练循环每消费完一块就调用本方法，推进 cursor。预取提前量 = cursor + lookahead，
// 默认 lookahead = 8（与 v3 §7.2 的"自适应预取"一致；命中率低于阈值自动调小）。
func (p *PersistentPrefetcher) OnConsumeSideAdvance(consumedIndex int64)

// OnStageSwitch 由 OrderSource 调用：阶段切换时把当前阶段未消费的块降级为
// 后台补全优先级（不取消在途请求），把新阶段的开头块序列作为新的启动窗口
// 高优先级入队。返回切换时刻到新窗口就绪的预估时间，供 Agent 上报。
func (p *PersistentPrefetcher) OnStageSwitch(newOrder ConsumeOrder) (estReadyIn
        time.Duration, err error)

// Status 返回当前阶段、cursor 位点、预取队列长度、预取命中率（用于自适应 K 调参）。
func (p *PersistentPrefetcher) Status() PrefetchStatus
```

v4 新增职责模块二：阶段切换重预取（Stage-Switch Reprefetch）

阶段切换事件源有三类（详细接口见 §6.4）：1. **epoch 边界**：DataLoader 钩子检测到当前 epoch 即将结束（消费完最后一个分片），上报”下个 epoch 的 seed 和顺序”；2. **shuffle 重排**：seed 变化（每个 epoch 默认都会 shuffle，但有些 Job 会固定 seed 跨 epoch 复用顺序）；3. **curriculum 阶段切换**：用户在 Job annotation 里声明 stage 列表，每个 stage 有独立的数据子集和顺序。

阶段切换时 Agent 做什么：- 接收新顺序（`OnStageSwitch`）；- 把旧阶段正在预取但未消费的块降级为后台补全优先级（避免浪费带宽，但也不抢新阶段的窗口）；- 按新顺序重新启动预取：开头 K 块走启动预取优先级（中优先级，高于持续预取但低于新 Job 启动），其余走持续预取；- 阶段切换的 stall 时间作为单独指标 `traincache_stage_switch_stall_seconds` 上报。

5.4 读侧信号与命中率口径（v4 把 inotify 同时用作消费顺序校准）

- 用 inotify 监听缓存目录的 `IN_OPEN` 事件（做去抖：同一个 pid 多次打开同一个文件合并成一次，采样率可配），由此得到文件的打开频率和顺序；
- **v4 新增**：inotify 观察到的实际打开顺序与 OrderSource 推送的预期消费顺序做交叉校准——如果实际打开顺序与预期顺序偏离超过阈值（默认 20%），PersistentPrefetcher 主动重新对齐 cursor 到实际位点，防止”预想顺序”和”真实顺序”长期漂移；
- **命中率口径要写进报告**：消费者侧——由插了桩的 dataloader 上报它打开的块集合，再和本地块集合求交；我们**不声称 Agent 在读路径上做了精确记账**（preheat 模型做不到这一点）；更进阶的替代方案是 eBPF（那是 S3 项目的内容）。

6. 核心 1：训练全程数据流加速闭环（项目灵魂，v3 → v4 升级）

> v3 的 §6 是”预热-调度闭环”，只解决启动阶段。v4 把它扩展为贯穿训练全程的闭环， 分三个子阶段：调度-启动预取（§6.2）、训练中持续预取（§6.3）、阶段切换重预取（§6.4）。 三者共用同一套缓存库（CAS、路径索引、fetch 抓取管线）、同一个 Agent 进程、 同一份消费顺序输入，只是优先级和触发时机不同。v3 的 gate/Score/降级/盈亏曲面 机制全部保留，得分公式从覆盖率改为前缀就绪度 + 启动窗口加权（§6.2 形式化）。

6.1 三阶段统一控制回路

```
Job 创建（声明 Dataset + 消费子集 + 启动窗口大小 K + 门控模式注解 + consumeOrderRef）
  → Controller 注入 SchedulingGate: traincache.io/preheat-pending
  → 按放置策略选候选节点（fanout=1 / 2-3 / 全部，本身是实验变量）
  → 向候选 Agent 下发"启动前缀块清单"预热任务（前 K 块，字节预算 + 速率上限）
  → Agent 块级进度回填 status：prefixLen、startupWindowReady
  → prefixLen ≥ K（启动窗口全就绪） → 摘除 gate
     或总超时（默认 5m，注解可覆盖）→ 降级：去 gate/去亲和正常调度
  → Score plugin 按"前缀就绪度 + 启动窗口加权"打分 → bind 到最佳节点
—— Pod 落地，进入训练中阶段 ——
  → PersistentPrefetcher.Start：按消费顺序把后续块以低优先级持续预取
  → DataLoader 钩子每消费一块上报 cursor 推进
  → 阶段切换事件到达 → OnStageSwitch：新顺序的开头 K 块走中优先级，旧阶段降级
  → 训练结束，Agent 把命中热度 dump 到磁盘（沿用 v3 inotify 热度统计）
```

实现时要注意这些点（保留 v3 踩过的教训并补 v4 新教训）：

- gate 的注入和摘除必须幂等，再加 finalizer 防止产生孤儿对象；**k8s 版本不低于 1.26**，kind 镜像版本在阶段 0 确认；
- **Score plugin 内部不允许做网络调用**：用 informer watch 聚合 status，配合本地缓存加 1 秒 TTL——把 scheduler 打挂是调度器扩展里的经典坑；
- **v4 新教训**：Score plugin 读 status 时要优先读 `prefixLen` 而非 `readyRatio`，否则 v3 那种”覆盖率虚高”的错位会复发；
- 打分以 `score(node) = α·prefix_ready_ratio + (1-α)·window_ready_ratio` 为主（形式化定义见 §6.2），节点空闲磁盘和在途预热负载作为次要排序依据；门控模式分两种：`required`（硬等，达不到阈值就只能等超时降级）与 `preferred`（软门控，超时立即放行）；
- 降级语义要设计成”去掉约束继续调度”，不能直接报错；降级时写入 Condition=Degraded、记录 Event 并上报 Prometheus 指标；数据源不可用的时候有总超时兜底，**Job 永远不会因为预热问题而死锁在 Pending 状态**；
- 预热速率受预算约束（默认不超过节点可观测空闲带宽的一定比例），防止预热点火的时候挤压在线 Job；
- **v4 新教训**：训练中持续预取的优先级永远低于新 Job 的启动预取——多 Job 共享一个节点时，新 Job 的启动预取要能打断老 Job 的持续预取，否则老 Job 的预取会拖延新 Job 的启动。

6.2 前缀就绪度形式化定义与得分公式（v4 核心修正）

记号：

- Job 声明的消费子集按消费顺序排列的块序列 `B = (b_0, b_1, …, b_{n-1})`；`|B| = n`；
- Job 的启动窗口大小 `K`（默认 `K = batch_size × prefetch_factor`，由 Job annotation 声明，未声明时取保守默认 `K = 32`）；
- 节点 `node` 在某个时刻的预热就绪块集合 `R_node ⊆ B`；
- 启动窗口 `W = {b_0, b_1, …, b_{K-1}}`（前 K 块，K 可能大于 n，此时 W = B）。

前缀就绪度（Prefix Readiness）：

```
prefix_len(node, Job) = max{ k ∈ [0, n] : ∀ i < k, b_i ∈ R_node }
                        （从第 0 块开始连续就绪的块数；空集时为 0）
prefix_ready_ratio(node, Job) = prefix_len / n
```

直觉：只有从开头连续就绪的那一段对启动有用；预热了第 90~99 块但第 50 块没到，`prefix_len = 50`（不算 90~99），覆盖率却虚高为 60%。

启动窗口就绪度（Startup Window Readiness）：

```
window_ready_ratio(node, Job) = |R_node ∩ W| / |W|
```

直觉：启动窗口里的就绪比例；当 `prefix_len ≥ K` 时本值 = 1。

综合得分公式：

```
score(node, Job) = α · prefix_ready_ratio(node, Job)
                 + (1 - α) · window_ready_ratio(node, Job)
```

- α ∈ [0, 1]，默认 `α = 0.7`（偏向前缀连续性，因为前缀就绪才能启动）；
- α 是 Job annotation 可覆盖的参数；α = 1 退化为纯前缀就绪度，α = 0 退化为纯启动窗口覆盖率；
- 次要因子：`free_disk(node)`、`inflight_prefetch_bytes(node)`（用作 tie-breaker）。

门控判定：

模式	放行条件（满足任一）
`required`	`prefix_len(node) ≥ K`（启动窗口前缀全就绪）；**或** 总超时触发，进入降级
`preferred`	`score(node) ≥ τ`（默认 τ = 0.9）；**或** 软超时（默认 1m）触发，立即放行

降级语义：

- 超时后摘除 gate，正常调度；

- 选 `score` 最高的候选节点 bind；若所有候选节点 `score` 都 < 0.3，允许 bind 到任意候选节点（让训练 Pod 自力更生回源读，等价于直读 MinIO 基线）；
- 写入 `Condition=Degraded`，记录 Event `PreheatTimeout`，上报指标 `traincache_preheat_degraded_total{dataset,job}`；
- **Job 永远不会因为预热问题死锁在 Pending**——这句话是设计底线。

与 v3 公式的对比（写在报告里的诚实清单）：

口径	v3	v4
度量主体	覆盖率 readyRatio	前缀就绪度 + 启动窗口加权
启动阶段语义	预热 60% 即可放行，但可能第 50 块没到 → 启动卡住	前缀必须就绪到启动窗口才放行
全程收益记账	只记启动一次	启动预取字节 + 持续预取字节 + 阶段切换重预取字节，分三笔
缺点	虚高得分导致启动 stall	前缀就绪的硬约束可能让候选节点都达不到阈值 → 触发降级更频繁（实测中要量化降级率）

6.3 调度-启动预取阶段（v4 保留 v3 主体，得分公式切换）

控制回路见 §6.1 上半段。要点：

- Controller 收到 Job 创建事件后，读取 Job annotation 里的 `consumeOrderRef`，从中解出 `B = (b_0, …, b_{n-1})` 与 `K`；
- 按放置策略选候选节点（fanout 是实验变量），给每个候选节点上的 Agent 下发”启动前缀块清单”（前 K 块）预热任务；
- Agent 完成一块就把 `prefixLen` 加 1 回填到 Dataset.status（带 finalizer 防止孤儿）；
- Controller 的 reconcile 逻辑必须幂等：informer 重放不应产生重复任务；
- 启动预取的字节预算受节点空闲带宽约束（沿用 v3），防止点火挤压在线 Job。

这一阶段对应 v3 的 §6.1 主体，差别只在得分公式与放行条件按 §6.2 切换。

6.4 训练中持续预取阶段（v4 全新）

这一阶段是 v4 项目灵魂——v3 的整套缓存库只服务了启动这一次，v4 把它扩展成贯穿训练全程。

设计目标：让缓存命中率在整个 epoch 内保持高位，消除中后期的 data stall。本质上，把”一次性预热”变成”持续性流式预取”。

消费顺序从哪里获取（v4 关键设计，三种来源按推荐度排序）：

1. **顺序生成器 sidecar（主路径，推荐）**：训练 Pod 里跑一个轻量 sidecar，按 Job 配置（`dataset`、`epochs`、`shuffle_seed`、`batch_size`、`prefetch_factor`、可选的 `curriculum_stages`）算出每个阶段的消费顺序，通过共享内存或 Unix socket 推给同节点的 Agent。优点：训练框架无侵入，顺序是确定性的可复现。缺点：sidecar 多占一点资源；

2. **DataLoader 钩子（精确路径，校准用）**：训练框架的 DataLoader 钩子（即 Agent 文档里”插了桩的 dataloader”）每次实际读块前上报下一块要读的 `(path, chunkIndex)`。优点：精确反映真实读取顺序，包括用户自定义的 collate/sampler 行为。缺点：要求训练代码加钩子，侵入性高，且有些训练框架的 sampler 行为复杂难还原；
3. **Job annotation 声明（最简路径，仅启动用）**：Job 在 annotation 里直接声明 `consumeOrderRef` 指向对象存储上一份预先生成的顺序文件。优点：零侵入。缺点：只能给出静态顺序，无法反映运行时变化（如 DataLoader 的 worker 间分配）。

v4 的取舍：主路径用方案 1（顺序生成器 sidecar），方案 2（DataLoader 钩子）作为校准信号——Agent 拿方案 1 算的预期顺序，跟 inotify 观察到的实际打开顺序比对，偏离超过阈值就重新对齐 cursor。方案 3 仅用于启动阶段的最小可行实现（步骤 5 的端到端 MVP）。

接口设计（Go，伪代码）：

```
// OrderSource 是消费顺序的来源抽象，Agent 持有一个实例。
// 实现可以是 sidecar 客户端、DataLoader 钩子客户端、或 annotation 解析器。
type OrderSource interface {
    // CurrentOrder 返回当前阶段的消费顺序（块引用列表）。
    // 列表可能很长，允许实现用 chunked iterator 返回，避免一次拉爆内存。
    CurrentOrder(ctx context.Context) (ConsumeOrder, error)
    // SubscribeStageSwitch 订阅阶段切换事件；返回 channel，每收到一个事件 push 一份新
        ConsumeOrder。
    SubscribeStageSwitch(ctx context.Context) (<-chan ConsumeOrder, error)
}

// ConsumeOrder 是某个阶段的完整消费顺序。
type ConsumeOrder struct {
    Dataset      string          // 数据集名
    SnapshotID   string         // 快照版本
    Stage        string          // 阶段标识：epoch-0、epoch-1、curriculum-stage-2 等
    ShuffleSeed  int64          // 本阶段 shuffle 种子
    Chunks       []ChunkRef     // 按消费顺序排列的块引用
    BatchSize    int            // DataLoader batch_size
    PrefetchK    int            // 启动窗口大小 K（已由 Job annotation 推导好）
}

type ChunkRef struct {
    Path  string `json:"path"`
    Chunk int64  `json:"chunk"`
}
```

预取策略：

- 预取提前量 `lookahead` 默认 8 块（与 v3 §7.2 自适应预取一致）；`OnConsumeSideAdvance` 推进 cursor 后，把 `[cursor+1, cursor+lookahead]` 范围的块以”邻域预取”优先级送进 fetch 抓取管线；
- 预取命中率低于阈值（默认 30%）就自动把 `lookahead` 调小到 4，再低到 2；高于 70% 调大到 16；这是 v3 自适应预取策略的延伸；
- 预取请求与显式启动预取共用 `fetch.Fetcher` 的 in-flight map（同一 key 自动合并），不会重复下载；

- **v4 新教训**：训练中持续预取的字节总量可能远大于启动预取（一个 epoch 可能消费数十万块），所以带宽预算管理必须把”全程预算”作为独立口径上报，不能跟启动预算混算。

6.5 阶段切换重预取阶段（v4 全新）

这一阶段处理 epoch 切换、shuffle 重排、curriculum 阶段切换时的缓存重新铺排。

事件源（来自 OrderSource.SubscribeStageSwitch）：

- **epoch 边界**：DataLoader 完成当前 epoch 的最后一个 batch 时上报”下个 epoch 的 seed 和顺序”；
- **shuffle 重排**：seed 变化即视为新顺序；seed 不变（用户跨 epoch 复用顺序）则不触发重预取；
- **curriculum 阶段切换**：用户在 Job annotation 里声明 `curriculum_stages` 列表，每个 stage 有独立的数据子集和顺序；sidecar 按 stage 推进边界上报。

Agent 在阶段切换时做什么：

1. 收到 `OnStageSwitch(newOrder)`；
2. 旧阶段在途预取请求：降级为后台补全优先级（不取消，已花掉的带宽不浪费）；
3. 旧阶段未消费的块（cursor 之后的）：从抓取队列里移除，腾出带宽给新阶段；
4. 新阶段的开头 K 块：作为新启动窗口，走”阶段切换重预取”优先级（中优先级，高于持续预取但低于新 Job 启动预取），送进抓取管线；
5. 新阶段的剩余块：新启动窗口就绪后，转入持续预取模式（low priority）；
6. 上报 `traincache_stage_switch_stall_seconds{dataset, job, stage}`：从切换事件到新窗口就绪的时长；
7. 上报 `traincache_stage_switch_wasted_bytes{dataset, job, stage}`：旧阶段在途但被取消的字节数（如果选择取消的话）——默认不取消，所以这个值通常为 0。

与持续预取的协同：

- 阶段切换不重启 PersistentPrefetcher 进程，只换它的消费顺序源；
- 切换瞬间允许 stall 短暂回升（预期行为，不算系统故障）；
- 切换完成后 stall 应重新回落到持续预取稳态水平——若没有回落，说明新阶段的预取跟不上消费，需要把 `lookahead` 调大或检查带宽预算。

6.6 超时降级（v3 保留 + v4 新增全程降级）

启动阶段降级（v3 保留）：总超时默认 5 分钟（注解可覆盖），超时后去 gate、去亲和、正常调度，写 Degraded 状态。

v4 新增全程降级：训练中持续预取长期跟不上消费（连续 30s 内 cursor 追不上消费位点）时，PersistentPrefetcher 进入”回源读”模式——停止预取，让训练 Pod 直接回源读 MinIO，等价于零缓存基线。这种降级不会停训练，但会显著降速。触发时上报 `traincache_prefetch_degraded_total{dataset, job}` 指标，并在 Condition 里写 `PrefetchStarved`。

6.7 预注册假设与盈亏平衡旗舰图（v3 → v4 扩展）

v3 的盈亏平衡曲面只看启动阶段。v4 把它扩展成三张图：

图一：启动阶段盈亏曲面（v3 保留）

- 三个维度：源站带宽、数据集/缓存容量比、fanout；
- 纵轴：启动阶段 makespan 减去自由调度基线的差值（负值=快，正值=慢）；
- 假设预注册：我们预期门控策略在多 epoch 重复训练、数据集大于缓存容量但热子集稳定、源站慢或带宽贵的场景下赢；在单次顺序 epoch、缓存容量远大于数据集、源站极快、预热频繁超时的场景下输。

图二：全程 makespan 盈亏曲面（v4 新增，**新旗舰图**）

- 三个维度：源站带宽、数据集大小、训练 epoch 数（1 / 3 / 10）；
- 纵轴：全程 makespan（从 Job 提交到最后一个 epoch 结束）相对 Fluid 基线的差值；
- 假设预注册：我们预期 v4 在多 epoch（≥3）场景下显著赢（持续预取让中后期 stall 不回升），在单次顺序 epoch 场景下与 v3 表现相近（持续预取没有发挥空间）；

图三：阶段切换 stall 时间曲线（v4 新增）

- 两个维度：epoch 数、shuffle 策略（固定 seed / 每 epoch 重排 / curriculum 多 stage）；
- 纵轴：阶段切换瞬间到 stall 回落稳态的时间；
- 假设预注册：我们预期 v4 在每 epoch 重排场景下 stall 时间显著低于 Fluid 基线（Fluid 没有阶段感知），在固定 seed 跨 epoch 复用顺序场景下与基线相近（无顺序变化，持续预取天然命中）。

假设要在实验开始之前写死（预注册，最后数字长什么样就报什么样）——禁止看到结果再调参”做出”目标数字。这三张图哪怕大半区域是负的也算成功——它们共同回答了”这东西什么时候不该用”。

配套次旗舰图：fanout 取 1/2/3 时，启动阶段 makespan 收益与重复预热字节成本的 Pareto 散点图（v3 保留）；全程 makespan 收益与持续预取总字节成本的 Pareto 散点图（v4 新增）。

7. 核心 2：欠配缓存下的准入与预取（v3 → v4 定位降级，保留主体）

> v3 把”准入策略对照”当作核心 2，v4 把它降为”次要模块”。理由：v4 的核心叙事是 “全程数据流加速”，准入策略只是其中一个优化点，不是项目灵魂。但 v3 的实验设计、 W-TinyLFU 规格、阴性结论接受态度全部保留不变。下面文字沿用 v3，只在末尾加一段 与 §6.4 持续预取的协同说明。

7.1 为什么 v3 的 W-TinyLFU 旗舰图可能是一条平线

全量顺序 epoch 加上缓存容量足够大时，LRU、LFU 和任何准入策略的命中率都会趋同——准入策略之间的差异，只有在**缓存欠配、访问倾斜、热子集被重复消费、多租户互相挤占**四个条件同时成立时才会显现。所以这个实验必须主动构造上述场景，不能在顺序扫描上干巴巴地比较三个算法。

7.2 实验设计

- **多 Job 倾斜 trace（负载 C 轨，§10.1）**：让 Job A 反复读取 10% 的热分片（模拟多轮微调和重复 epoch），Job B 顺序扫描全量数据（流寇式扫描），缓存容量设为数据集的 30%；
- 对照组：在 LRU、LFU、W-TinyLFU 三种策略下，分别记录 **Job A 的命中率与 stall 占比**，以及扫描作业把热集驱逐出去的次数；

- W-TinyLFU 的实现（保留 v3 规格）：window（SLRU 的 probationary/protected 两段）加 count-min sketch（4 行、宽度取缓存块数的 10 倍、3bit 饱和计数）加 doorkeeper bloom；准入规则是 `freq(candidate) > freq(victim)` 时才换入；定期做 halving；参数（window 占比、halving 周期）扫点画出敏感性图；
- **自适应预取**：inotify 观测到块 i 被打开，就抓取后继的 K=8 个块；如果预取命中率低于阈值，就把 K 降下来；
- **允许得到的结论**：如果全量顺序场景下三种策略没有差异，而倾斜场景下有差异，那就如实写”准入策略对单 Job 的顺序训练并不关键，真正关键的是预取调度与跨租户保护”。这是一个成熟的阴性发现，算不上实验失败。

7.3 与 §6.4 持续预取的协同（v4 新增）

v4 的持续预取本身已经在做”按消费顺序提前抓取后续块”，准入策略决定的是”缓存不够大时谁留下”。两者关系：

- 持续预取抓回来的块，进入缓存时受准入策略裁决——如果 W-TinyLFU 认为这块的频率不够高，直接拒绝写入（让训练 Pod 回源读）；
- 持续预取的 `lookahead` 越大，预取块集合越大，准入策略越重要（小 lookahead 时准入几乎无关）；
- 报告里要单独画一张图：`lookahead × 准入策略` 二维命中率曲面，说明两者如何相互影响。

8. 支撑臂设计（v3 保留，无改动）

8.1 S3 合并回源网关（保留 v3）

- 用 `singleflight` 按对象 key 合并并发的 GET 请求；大对象通过 `io.Pipe` 或 splice 流式扇出，禁止整块缓冲之后再分发；
- AIMD 背压：当 MinIO 的 P99 延迟上升或出现 5xx 时，令牌桶减半，平稳期再线性恢复；并发上限本身就是”对中心存储的礼貌”；
- 用 `sync.Pool` 复用块级 buffer，并通过 `GODEBUG=gctrace=1` 对比每字节分配数和 GC pause（作为 Go GC 调优的热身）。

8.2 P2P 块共享：在 CE 基线上复现 EE”缓存组”语义（保留 v3）

v4 不改 v3 设计。要点：节点发现复用 CRD status；一致性哈希环定”老家节点”；拉取顺序优先找 peer，最多同时向 2–3 个 peer 多源拉取，peer 都没有再回源；peer 请求失败立即穿透回源；全局带宽预算用令牌桶，origin 保底占 20%；传输层抽象为 `Transport` 接口，gRPC stream 默认，RDMA 实验臂挂接口后面。

v4 与持续预取的协同：持续预取产生的抓取请求也走 P2P 路径——同节点的 peer 有要的块就不回源；这是 v4 在 P2P 层新增的语义，但不需要新接口，只是 `fetch.Fetcher.Fetch` 内部走的查找链路多了一层 peer 询问。

8.3 Checkpoint 异步 sidecar（保留 v3，语义错误已修正）

v3 已修正 v1 的语义错误：训练恢复时权重和 optimizer state **都完整之后才允许跑第一个 step**。v4 不再改。

8.4 混沌套件（v3 四件套 + v4 新增一件）

注入	手段	期望
Agent kill -9	让进程崩溃再重启	重扫描后恢复索引，没有数据错误；**v4 新增**：PersistentPrefetcher 重启后从 dump 的 cursor 恢复，不重新启动预取（除非用户主动）
网络劣化	tc/netem（仅在特权测试容器里使用）	背压生效，P99 恶化但有界；**v4 新增**：持续预取进入回源读模式，训练 Pod 不死
位翻转	通过测试钩子篡改块字节	校验失败 → 重新拉取 → 有指标记录
数据源宕机	把 MinIO 缩容到 0	门控超时后降级，Job 不会卡死；**v4 新增**：持续预取进入 starved 状态，训练 Pod 退化但不停
v4 新增：阶段切换事件丢失	杀 OrderSource sidecar	Agent 30 秒收不到 cursor 推进就降级为 LRU 通用预取（用 inotify 观察顺序兜底），上报 `traincache_order_source_lost_total`

9. 调研/实验臂（只做实验或只写文档，绝不进主路径）

> v3 这一节保留不变。RDMA 实验臂、io_uring、版本化、GPU 侧技术知识边界四块都不动。 唯一与 v4 相关的注释：io_uring 实验臂对小文件高 IOPS 的预期收益不变，但 v4 的持续预取 在小文件场景下会让 syscall 数量进一步上升（预取本身就是高频小请求），io_uring 的收益 可能在 v4 的持续预取路径上比 v3 的启动预取路径上更明显——这是一条值得扫点的实验变量。

9.1 RDMA 传输臂（保留 v3）

9.2 io_uring / S2（保留 v3，加 v4 持续预取路径上的预期收益注释）

9.3 版本化（保留 v3）

9.4 GPU 侧技术的知识边界（保留 v3）

10. Benchmark 方法学（v4 主要更新：分启动阶段与全程阶段两份报告）

10.1 三轨负载（v3 保留 + v4 加 D 轨）

轨	形态	定位
A 元数据压力臂	100 万个 4KB 小文件（脚本生成，参数可调）	模拟元数据风暴和小文件 IOPS，是 io_uring 的战场；不再自称训练主流
B 真实分片主臂	webdataset tar 或 Mosaic MDS，百 MB 分片加真实 PyTorch DataLoader，配合确定性 shuffle；脚本合成真实 JPEG 编码的分片	对应 2026 年训练的真实形态，所有旗舰数字都以这一臂为准
C 多 Job 倾斜臂	Job A 反复读 10% 热分片，Job B 全量扫描，缓存欠配到 30%	核心 2 专用
D 多 epoch 切换臂（v4 新增）	单 Job 连续跑 10 个 epoch，每 epoch 用不同 seed 做 shuffle，缓存欠配到 50%	**核心 1 全程收益专用**：测阶段切换 stall、持续预取命中率时间序列、全程 makespan

模拟器只用来做参数扫点；旗舰数字必须有真实 dataloader 复跑。CPU 训练循环（不挂模型或只挂极小模型）以 dataloader stall 占比作为代理指标。

B 轨数据合成纪律（零下载、真解码）：我们不用纯随机 tensor，也不去下载真实的 ImageNet；用脚本批量生成随机像素图像，做真实的 JPEG 编码（PIL 或 turbojpeg，质量和分辨率参数化），再按 webdataset 规范打成百 MB 的 tar 分片，固定随机种子。报告里要注明”合成数据，I/O 与解码路径真实，语义内容随机”。

10.2 在笔记本上造出真实的冷热差（v3 保留）

单机加一块物理盘，天然不存在”本地 NVMe 对比网络取数”的速度差。解决办法是用 `tc qdisc tbf/netem` 对 Agent 到 MinIO 的源站路径做受控限速和延迟注入（比如 100Mbps、20ms 这一档）。命中本地缓存的读请求根本不会走这条路径。

10.3 对照组矩阵（v4 把 v3 的五组扩展为按阶段报告）

v3 的五组对照（直读 MinIO / JuiceFS CE 冷读 / JuiceFS warmup 热读 / Fluid 全家桶 / TrainCache 三策略）保留。**v4 新增报告口径**：每组实验都要分别报告启动阶段指标和全程指标，如下表。

指标	启动阶段（Pod 落地前）	全程阶段（Pod 落地后到训练结束）
makespan	Job 提交 → 第一个 step 开始	Job 提交 → 最后一个 epoch 结束
stall 占比	第一个 step 的 wait_time/step_time	全程所有 step 的 wait_time/step_time 时间序列
缓存命中率	启动窗口命中率	分阶段命中率曲线（启动期 / 稳态期 / 阶段切换期）
字节成本	启动预取字节 × fanout	启动预取 + 持续预取 + 阶段切换重预取，分三笔

10.4 硬纪律（v3 保留 + v4 加一条）

1. 每组实验**至少跑 7 次取中位数，并报告 bootstrap 95% CI**；只跑 3 次取中位数的结果不进报告；
2. 每轮实验之间清页缓存（`sync; echo 3 > drop_caches`）并重启相关进程；
3. 环境完全锁定：镜像 digest、JuiceFS/Fluid 版本、tc 参数、负载配置、随机种子全部入库，做到 `make bench COND=xxx` 一键复现；
4. 先写预注册假设（§6.7），禁止看到结果之后再调参”做出”目标数字；目标百分比从正文删掉；
5. **v4 新增**：全程指标的报告中必须明示”持续预取字节成本”——不能只报 makespan 收益而隐藏预取带来的回源字节代价，否则就是 v3 那种”只报赢家”的错误复发。

11. 项目优点（v4 更新）

1. 完整的 Operator 工程：CRD（在 etcd 尺寸约束下的 shard 状态设计）、幂等 reconcile、condition/finalizer；
2. 调度器框架的全链路：SchedulingGate 加 Score（无网络调用的本地缓存范式）、**前缀就绪度加启动窗口加权的得分公式**、超时降级与死锁防护；
3. **训练全程数据流加速闭环**：把一次性启动预热扩展成贯穿训练全程的三阶段闭环（调度-启动预取 / 训练中持续预取 / 阶段切换重预取），这是 v4 相对 v3 的核心修正，也是相对 Fluid 的真实缺口；
4. 双层标识模型（路径索引加 CAS 存储层）带来的内置校验与安全去重，以及”纠正自己术语错误”的真实设计过程；
5. 把缓存准入放在**正确的场景**下对照（欠配、倾斜、多租户），并且能讲清楚它在什么时候无效；
6. I/O 与分布式方面的硬货：singleflight 合并加 AIMD 背压、P2C 带宽预算、CAS 下 unlink/rename 的内核语义、混沌五件套（v3 四件套 + v4 阶段切换事件丢失）；
7. 与真实的 Fluid/JuiceFS 做了五组对照，**分别报告启动阶段与全程阶段指标**，改进最终回馈给 Fluid——第三方背书是简历上信用最高的东西。

12. 诚实清单（v4 更新）

- 3 节点的 kind 集群只能用于机制验证和相对趋势分析，不做千卡规模的外推；
- 单物理盘上的冷热差是用 tc 受控限速造出来的，绝对带宽数字最终要靠一次双盘云主机校准来替换；
- preheat 模型的命中率是旁路口径（inotify 加消费者打点），并不是读路径上的精确记账；
- **v4 新增**：消费顺序来自顺序生成器 sidecar 的”预期顺序”，与真实 DataLoader 行为可能有偏离——inotify 交叉校准只能发现偏离，不能完全纠正（DataLoader 的 sampler 行为复杂时尤其如此）；
- **v4 新增**：阶段切换 stall 时间的测量依赖 OrderSource 主动上报，若 OrderSource 自身崩溃，stall 时间测不准（v4 用 30 秒无 cursor 推进作为兜底信号，但兜底期间的真实 stall 不可精确计量）；
- 核心 1 的”空白”判断以阶段 0 的现勘为准（包括探针 C 验证训练全程 stall 真实存在），如果闸门反转，叙事就收缩，不硬撞；
- 没有真 RDMA/GDS 的性能数据：Soft-RoCE 只用来验功能，eRDMA 只做一次性的按小时校准；
- 元数据依赖 etcd，不自研 Raft：这是刻意的取舍，底座能力由 S1 补上；
- FUSE 与自研失效协议明确不做，不虚报。

13. 砍项清单（v3 → v4 变更摘要）

v3 内容	v4 处置	理由
核心 1 = 预热-调度闭环	**扩展为三阶段闭环**（启动预取 + 持续预取 + 阶段切换重预取）	v3 只覆盖启动阶段，缓存库只用了启动这一次；v4 把项目灵魂升级为全程加速
得分公式 = 覆盖率 readyRatio	**改为前缀就绪度 + 启动窗口加权**	覆盖率口径下预热第 90 块但第 50 块没到时得分虚高，启动照样卡住；前缀就绪度才是启动可用的真实度量
readyRatio 作为门控依据	**prefixLen ≥ K 才放行**（required 模式）	与得分公式对齐
旗舰图 1 张：启动盈亏曲面	**扩展为 3 张**：启动盈亏曲面、全程 makespan 盈亏曲面、阶段切换 stall 曲线	v3 只看启动，v4 三阶段分别量化
基座 Agent 职责	**追加两个新职责模块**：持续预取 + 阶段切换重预取	v3 的 Agent 只服务启动，v4 全程都要做事
核心 2 准入策略	**降级为次要模块**（实验设计不变）	v4 的核心叙事是全程加速，准入只是其中一个优化点
混沌四件套	**加一件**：阶段切换事件丢失	v4 新增 OrderSource 是新依赖，必须验证它的故障降级
五组对照报告	**每组分别报告启动阶段与全程阶段指标**	v3 只报全程 makespan，v4 必须分阶段才能体现持续预取的价值
三轨负载	**加 D 轨**：多 epoch 切换臂	v4 全程收益专用场景

14. 实施步骤（v4 重新编排，14 步对应新核心）

步骤	内容	验收 / 注意点
0 现勘	§4：4.0 可行性探针（含 v4 新增探针 C：训练全程 stall 真实存在）+ 部署 kind、MinIO、JuiceFS CE、开源 Fluid，跑通 DataLoad/亲和 demo + 跑 3 个 epoch 观察 Fluid 有无持续预取，取证注入内容与时序，产出现勘报告	**三闸门**：探针 A/B/C 亮绿灯 + 缺口被证据确认 + 探针 C 确认训练中 stall 真实存在，才能进入核心 1 三阶段闭环叙事
1 实验装置	tc tbf/netem 限速规程 + drop_caches 规程 + 四轨数据集生成器（A/B/C/D）	`make bench-up`；限速档位可扫点，冷态可重复；**D 轨固定 10 epoch、shuffle seed 每轮换**
2 缓存核心库	manifest 路径索引 + CAS 存储层（流式摘要/refcount）+ 限并发抓取	**已完成**：M1–M7 全部里程碑通过，54 测试全绿，接口冻结
3 Agent 服务化	gRPC（预热/状态）+ /metrics + 启动重扫描 + inotify；**v4 新增**：顺序生成器 sidecar 接口、PersistentPrefetcher 骨架、OrderSource 抽象	指标带 dataset 标签以防基数爆炸；重启恢复要有测试；**v4 新增测试**：OrderSource 失联 30s 后降级为 LRU 通用预取
4 数据面	DaemonSet + hostPath 只读约定 + 预热 CLI；**v4 新增**：消费顺序生成器 sidecar 部署模板、Unix socket 通信通道	手动预热之后，普通 Pod 可以全量读取；**v4 新增**：sidecar 启动后能向 Agent 推送一份 ConsumeOrder
5 Controller + CRD + 真实 dataloader	reconcile 分发与回填；webdataset/MDS + PyTorch DataLoader 跑通 B 轨；**v4 修正**：得分公式用前缀就绪度、门控用 prefixLen ≥ K	端到端 MVP；informer 重入幂等；真实负载在此接入
6 基准框架与 Fluid 上秤	五组对照自动化（至少先打通 1/2/3/5 组），≥7 次 + CI，一键出图；**v4 新增**：每组分别报告启动与全程指标	第一个简历成果：包含对手的对照报告
7 核心 1 三阶段闭环	**v4 重新定义**：调度-启动预取（gate + Score + 前缀就绪度 + 候选放置 + 超时降级）+ 训练中持续预取（PersistentPrefetcher）+ 阶段切换重预取（OnStageSwitch）	**三张旗舰图**：启动盈亏曲面、全程 makespan 盈亏曲面、阶段切换 stall 曲线；plugin 内无网络调用；PersistentPrefetcher 重启恢复有测试

步骤	内容	验收 / 注意点
8 核心 2 准入预取	C 轨多 Job trace；LRU/LFU/W-TinyLFU + 自适应预取；**v4 新增**：与持续预取的协同实验（lookahead × 准入策略二维命中率曲面）	倾斜场景差异曲线 + 顺序场景阴性结论 + 参数敏感性 + 协同图
9 S3 网关	singleflight + io.Pipe 扇出 + AIMD	大对象扇出时内存不涨
10 Checkpoint sidecar	multipart + manifest 发布点 + 修正后的恢复语义	训练恢复时状态必须完整；恢复时长有拆解
11 P2P 缓存组	哈希环 + peer 回源 + 带宽预算 + Transport 接口	源站出口字节下降；origin 保底生效；**v4 验证**：持续预取请求走 peer 路径不回源
12 混沌套件	§8.4 **五件套**全部通过（v3 四件套 + v4 阶段切换事件丢失）	恢复时间指标齐备；OrderSource 失联降级有指标
13 真值校准	两台带独立数据盘的 CPU 云主机 + tc 校准，五组对照全量复跑；**v4 新增**：D 轨 10-epoch 全程 makespan 真值校准	用真存储数字替换单机数字，两个版本都保留并说明差异
14 上游回馈	拿着 §6.7 的三张图向 Fluid 提 issue → 讨论 → PR（调度/DataLoad 方向 + 持续预取方向）	至少 1 个带数据的 issue；代码 PR 按维护者的认可程度推进

RDMA 实验臂（§9.1）挂在步骤 11 之后，按兴趣和预算推进，不占关键路径；io_uring（S2）在步骤 8 前后借助 A 轨数据独立推进。

15. 排期与死线（v4 倒推，与 P2 共用日历）

- **2027 年 2 月实习投递包的最低构成**：完成步骤 0–7，并在 Fluid 上提出至少 1 个带测量数据的 issue。也就是说，要有基座加三阶段闭环核心 1（五组对照至少跑完三组，再加上**三张旗舰图**：启动盈亏曲面、全程 makespan 盈亏曲面、阶段切换 stall 曲线）；
- **2027 年 8 月秋招包**：完成步骤 0–11、13、14（issue 有维护者的正式回应，或者 PR 已有实质进展）；
- 砍项顺序（时间不够时从后往前砍）：RDMA 臂 → FUSE → 混沌里的位翻转用例与阶段切换事件丢失用例 → P2P 的高级部分（多源调度）→ W-TinyLFU 的 Count-Min 完整版 → **持续预取的自适应 lookahead 调参**（保守默认值已足够进简历）；**核心 1 三阶段闭环、B 轨真实负载、Fluid 对照这三样任何时候都不砍**；
- 阶段 0 与 P2 现勘共用同一个”使用阶段”，立即启动。

16. 预期成果（v4 更新）

交付物：TrainCache 仓库（代码、README、ADR、复现命令）、《P1 对手现勘报告》（含探针 C）、五组对照实验报告（**分启动阶段与全程阶段两份**，含三张旗舰图与阴性区间）、Grafana 看板、两篇博客（一篇讲三阶段闭环全程实测、一篇讲 W-TinyLFU 在什么场景下才有用）、Fluid issue/PR 链接。

简历条目草稿（V4 主版）：

> 设计并实现了一套 K8s 原生的训练数据全程加速系统：在 v3”启动预热-调度闭环”基础上扩展为三阶段闭环（调度-启动预取、训练中持续预取、阶段切换重预取）；基于 CRD/Operator、SchedulingGate 门控，按 Job 数据子集的消费顺序的**前缀就绪度**（而非覆盖率）做门控与调度打分，超时降级；存储上采用路径索引加 CAS 内容摘要存储层（内置校验与安全去重）；Agent 在 Pod 落地后按消费顺序持续预取后续块，在 epoch/shuffle/curriculum 切换时按新顺序重新预取；在 3 节点 kind 集群下与真实的 Fluid/JuiceFS 做了五组对照，**分别报告启动阶段与全程阶段指标**，以全程 makespan 给出预取-调度闭环相对 Fluid 自由调度的盈亏边界（源站带宽 × 缓存欠配 × fanout × epoch 数），在多 epoch 重复训练场景下全程 makespan 下降 X%，同时量化了它在单次顺序扫描下的阴性区间；在缓存欠配的多租户场景下实测了 W-TinyLFU 与 LRU 的准入差异；相关测量结果以 issue/PR 的形式回馈给 Fluid 社区。